CS5013 Design Document: Unified Lab Project & Idea Tracking Platform

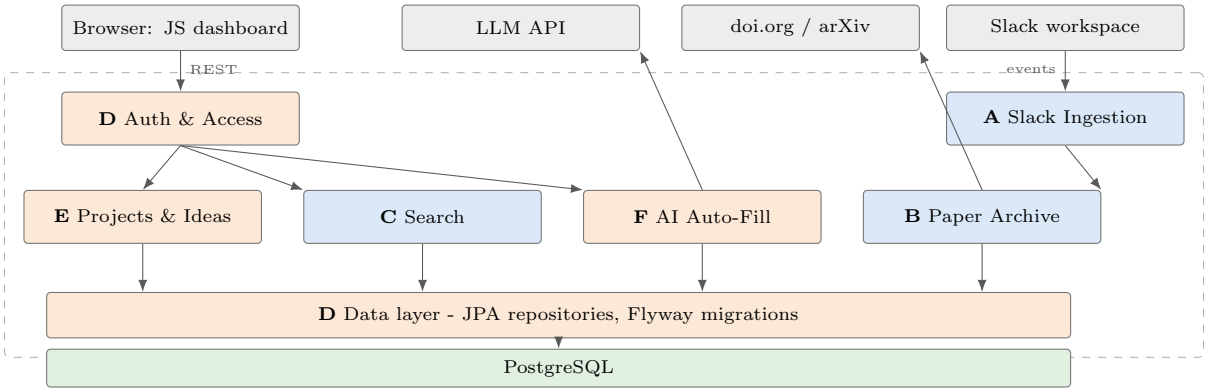
Team 47: Priyal Patel (CS24B047) & Sujal Jayant Kumar (DA24B026)

September 11, 2026

1 Architecture Note

The application is a client-server setup: a JavaScript dashboard in the browser, one Spring Boot server process, and one PostgreSQL database. A single process is sufficient for a lab of under fifty people.

- **Frontend (JavaScript):** A dashboard that communicates with the backend via REST* API*s, avoiding the full-page reloads of the current tracker. It carries two workflows. The *Papers Tab* has no forms; it fills automatically when someone drops a link in Slack. The *Project Tracker* takes manual input, under a "one required field per form" rule that removes the long mandatory checklists of the old system.
- **Backend (Spring Boot):** Controllers handle REST requests and Slack webhooks. The service layer holds the extraction logic, arranged as a fallback ladder: a DOI* or arXiv ID is resolved against the standard registries first, a page's citation meta tags second, and for a bare PDF an LLM* call via Java HttpClient extracts the title and authors. A link that survives all three is flagged for manual correction rather than dropped silently.
- **Database (PostgreSQL):** Stores projects, updates, ideas and paper links, with built-in full-text search in place of an external search engine. A unique constraint on the channel and message timestamp means a retried Slack event cannot create a duplicate row.
- **External services:** The Slack API (via Slack Bolt) for ingestion, the standard registries (`doi.org`, arXiv) for paper metadata, and an LLM API for auto-fill and the PDF fallback.



Dashed box = Spring Boot process, deployed by Module G; grey = systems outside it. Priyal: A, B, C. Sujal: D, E, F.

2 Module Split

Seven modules. Six are owned individually, three each; the seventh is shared.

Priyal Patel

- **Module A: Slack Ingestion.** Listens to the lab's Slack channels, verifies that requests genuinely came from Slack, and pulls out paper links in whatever form they appear. Routes each message to the database with the permalink attached, so every archived paper links back to the person who posted it.
- **Module B: Paper Archive.** Processes the links caught by Module A. Runs the resolution ladder - registries such as Crossref and arXiv first, then citation meta tags, then an LLM for bare PDFs - and stores the title, authors and the method that produced them.
- **Module C: Search & Paper Views.** Full-text search across projects, ideas and papers, and the dashboard pages that display the paper archive and search results.

Sujal Jayant Kumar

- **Module D: Auth & Data Layer.** Google sign-in with an allowlist for Raman Lab members, the PostgreSQL schema, and database migrations. Exposes the repository interfaces every other module relies on.
- **Module E: Projects & Ideas.** REST endpoints and dashboard elements for creating projects, logging ideas and viewing timelines. Enforces the "one required field" rule.
- **Module F: AI Auto-Fill.** Sends a free-text description to the LLM and fills in project title and tags from the result. Validates the returned JSON and presents it as a suggestion for confirmation, so nothing is saved without the user seeing it.

Both

- **Module G: Deployment & Handoff.** Packages the application as a single jar with its configuration held outside it, sets it up on whichever machine the lab settles on, and writes the README and install steps. Whether that machine is a department server or a paid always-on host is the lab's decision and a cost they choose to carry or not; the jar runs on either.

3 Test Plan

Database tests run against a real PostgreSQL instance so that constraints are exercised rather than mocked.

- **Module A (Slack Ingestion):** A dedicated test channel in the lab's workspace, fed diverse link formats (arXiv, DOI, PDFs). Links are picked up, irrelevant chat is ignored, and replayed events do not create duplicate records.
- **Module B (Paper Archive):** The resolution ladder against recorded API responses. A known DOI pulls exact registry data; a bare PDF triggers the LLM fallback and saves with the `source = MODEL` tag.
- **Module C (Search & Paper Views):** A mock dataset of 15–20 projects. A keyword present across a project title, an idea and a paper returns all three.
- **Module D (Auth & Data Layer):** Only allowlisted emails reach the system. Database constraints allow a project to save with just a title.
- **Module E (Projects & Ideas):** UI rendering and API response times, confirming pages load in well under half a second.
- **Module F (AI Auto-Fill):** Synthetic unstructured text with missing fields and special characters. With the LLM disconnected entirely, every form still works.
- **Module G (Deployment & Handoff):** A smoke check against the deployed instance - health endpoint, sign-in page, and a page requested after thirty idle minutes returning without a cold-start delay.

4 Revised Milestone Plan

What changed since the proposal

- **An integrated search bar for searching papers, ideas, and projects,** with a filter to filter out the relevant types.
- **Metadata is registry-first, not model-first,** and link handling is DOI-first rather than arXiv-first. The proposal talked mainly about arXiv, but the lab shares Springer, ACM and IEEE links too, and the DOI is what puts all of them through one code path. A model call is kept for bare PDFs, where nothing else works.

Block 1 - now to early October (build the core). Slack ingestion working end to end (Module A), starting in a test channel. Sign-in, the schema, and the project and update endpoints (Modules D and E), with enough of the dashboard to see a project timeline. By the end of this block a link posted in Slack appears in the platform.

Mid-demo, 9 October. The core workflow runs locally - slack fetches links and we can auth and add projects to the app. Known gaps at that point: auto-fill, search and hosting.

Block 2 - October (make it useful). Reference retrieval and the paper archive pages (Module B), then search (Module C). Idea logging finished and auto-fill added (Module F). If the Slack webhook is not usable by then, the polling fallback is written here instead.

Block 3 - last two weeks. Module G: deployment, README and install steps, and a dry run with Sabdhayini on her own machine. The block is reserved for this because the final demo is run by the stakeholder.

Final submission, 6 November. Stakeholder-run demo, test results and viva.

Future scope. Speech-to-text on the description field, so a project or idea can be logged by speaking it rather than typing. The transcript feeds the same Module F pipeline, so nothing downstream changes.

5 Risks and Plan B

- **Risk:** The Slack webhook fails to trigger reliably, through campus network restrictions or the app lacking workspace permissions.
  Plan B: A cron job in Spring Boot polls the Slack API for new messages instead of relying on event-driven webhooks. The cost is a few minutes of real-time freshness, which the stakeholder has confirmed is acceptable.
- **Risk:** Department server access is delayed or denied, and the lab does not want to pay for hosting.
  Plan B: Module G's output is a single jar with externalised configuration, so it runs on any always-on machine, including a lab member's own. What it will not run on is a free tier that sleeps, since that reproduces the cold-start problem being fixed.
- **Risk:** The stakeholder is uncomfortable sending unreleased project ideas or paper PDFs to a commercial LLM API.
  Plan B: The LLM call sits behind an interface, so it can point at a local open-source model on department hardware, or be switched off. Auto-fill and the PDF rung are lost; the registry and meta-tag rungs continue working.
- **Risk:** A link resolves to nothing, or the model rung reads a PDF wrongly. Reference lists are patchier still, since registries hold them only when the publisher deposited them.
  Plan B: The link is stored regardless, marked `UNRESOLVED` or flagged `source = MODEL`, and surfaced in the UI so the user can correct the details.

---

[*] API - Application Programming Interface; DOI - Digital Object Identifier; LLM - Large Language Model; REST - Representational State Transfer.